



Dossier de spécification

Projet Fil Rouge 1

2025-2026

Projets d'intégration des compétences acquises en 3A SRI

Auteurs :

Abdelhafidh Ghouilem

Arthus Doriath

Joan Belusca

Victor Chalumeaux

Historique des versions

Version	Date	Auteur	Description des modifications
1.0	25/10/2025	Groupe 1	Première version dossier livré le 26/10/2025

Table des matières

1. Introduction	4
1.1. Contexte du projet	4
1.2. Objectif général	4
1.3. Objectifs du dossier de spécification	4
2. Cas d'utilisation général	5
2.1. Effectuer des déplacements	5
2.2. Trouver des objets de couleur	6
3. Structure du système	7
3.1. Module d'interprétation et d'exécution des commandes	8
3.2. Gestion des actionneurs et environnement de test	8
3.3. Gestion du traitement d'image	9
3.4. Gestion des paramètres et de l'historique	9
4. Structure détaillé du système	9
4.1. Mode de contrôle	9
4.2. Traitement des requêtes utilisateur	10
4.3. Traitement des images	12
4.4. Traitement des déplacements	12
4.5. Simulation	13
4.6. Contraintes techniques	13
5. Planning	14
6. Annexes	15

1.Introduction

1.1.Contexte du projet

Le Projet Fil Rouge est un projet réalisé en troisième année de la filière SRI à l'UPSSITECH. Il s'étend sur l'ensemble de l'année universitaire et se divise en deux parties : le PFR1, mené au premier semestre, et le PFR2, réalisé au second.

Ce projet vise à simuler un environnement de développement professionnel en équipe, en intégrant à la fois des compétences techniques, organisationnelles et de communication.

Le corps enseignant joue le rôle de client, guidant les équipes dans la définition et la priorisation des besoins. Dans le cadre de notre projet, M. Julien Vanderstraeten assure cette fonction, en validant les choix techniques et fonctionnels tout au long du développement.

1.2.Objectif général

Le projet fil rouge a pour objectif de concevoir un robot nommé **Upssibot** qui doit être en mesure de recevoir des requêtes vocales ou textuelles, de les interpréter, d'effectuer des déplacements simples, et de réaliser une analyse d'image pour identifier les objets présents dans son environnement. Ces fonctionnalités seront validées dans un environnement Linux, en s'appuyant sur la bibliothèque Turtle pour la représentation graphique.

Le PFR1 se concentre alors exclusivement sur la conception du cœur logiciel et du mode manuel: il s'agit de développer des modules fonctionnels de base qui seront réutilisés et intégrés dans une version physique du robot lors du PFR2. Cette première phase privilégie donc la rigueur du code, la structure logicielle et la collaboration entre membres de l'équipe, dans un contexte proche de celui d'un projet professionnel.

Le besoin principal est de développer un robot capable d'interagir avec l'utilisateur de manière intuitive, tout en conservant une structure modulaire et extensible. Le système doit ainsi combiner trois capacités majeures : comprendre les commandes d'un utilisateur, agir en conséquence dans un environnement de simulation (PFR1), et percevoir son environnement à travers l'analyse d'images.

1.3.Objectifs du dossier de spécification

Ce dossier a pour but de présenter les spécifications fonctionnelles et techniques du projet **Upssibot**.

Il définit les besoins auxquels le système doit répondre, les contraintes associées, ainsi que la structure prévue pour sa mise en œuvre.

Ce document servira de référence commune à l'équipe et au tuteur tout au long du développement, afin d'assurer la cohérence entre les objectifs fixés et les solutions mises en place.

2. Cas d'utilisation généraux

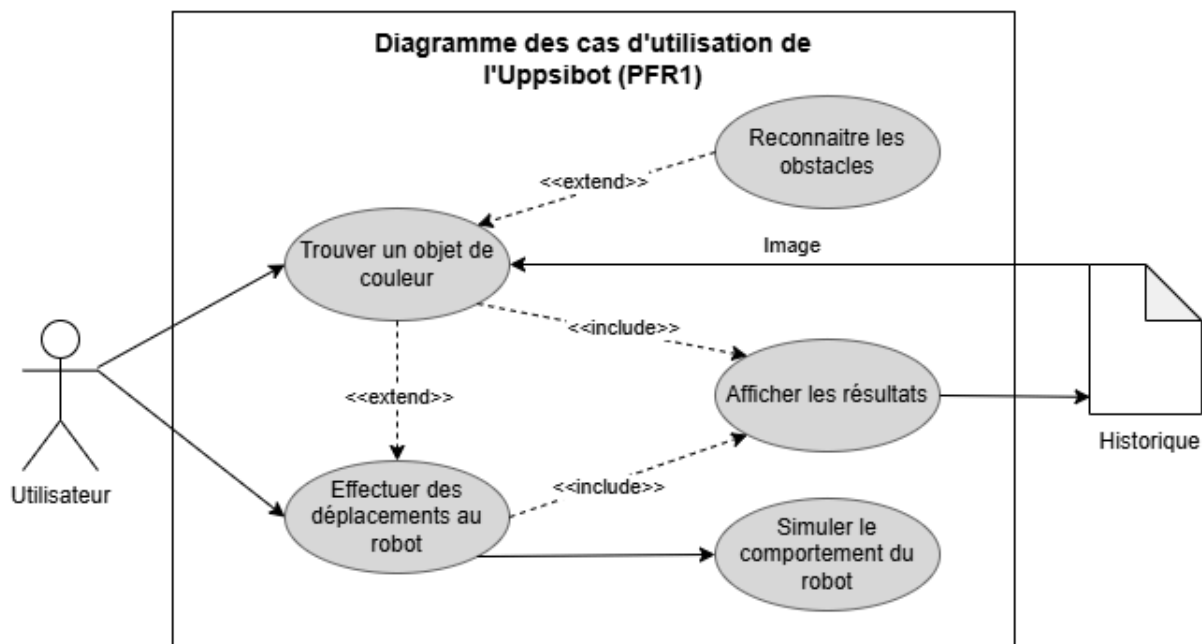


Figure 1: Diagramme des cas d'utilisation de l'Upsibot

La **Figure 1** ci-dessus représente le diagramme des cas d'utilisation de l'**Upsibot**, indiquant les interactions utilisateur-Robot. L'utilisateur peut à l'aide de commandes, faire effectuer des déplacements au robot ainsi que lui demander de trouver des objets sur une image sélectionnée.

Ces commandes peuvent être vocales ou textuelles. Elles appellent d'abord un module s'occupant du traitement d'image suivi d'un module chargé d'effectuer les déplacements.

2.1. Effectuer des déplacements

Les requêtes sont interprétées puis traduites en actions de mouvement que le robot exécute dans un environnement de simulation Python (*Turtle*) pour l'instant. Cette interface permet d'observer visuellement le déplacement et de valider le comportement attendu.

Cas nominal:

L'utilisateur envoie une requête vocale ou textuelle correctement interprétée, le robot exécute la commande correspondante, et l'interface Turtle ainsi que l'historique confirment la réussite du déplacement.

Cas dégradé:

1-Commande non reconnue : Le robot ne parvient pas à interpréter la requête. Aucune action n'est effectuée et un message « *Commande non reconnue* » est affiché sur l'interface et est enregistré dans l'historique.

2-Objet non trouvé : La commande est interprétée, mais elle implique un déplacement dépendant d'un objet qui n'a pas été trouvé par le module de vision. Le robot reste immobile et « *Déplacement impossible car l'objet est indétectable* » est affiché sur l'interface et est enregistré dans l'historique.

3-Obstacle détecté : La commande est interprétée, mais un obstacle est identifié par le module de vision. Le robot reste immobile et renvoie « *Déplacement impossible dû à un obstacle* » sur l'interface et dans l'historique.

2.2.Trouver des objets de couleur

Pour cela on utilise le module de vision afin d'identifier les objets présents dans l'environnement et leurs caractéristiques (couleur, forme, taille relative).

Certains objets peuvent être considérés comme des obstacles s'il y a un risque de collision, influençant ainsi le comportement du robot.

De plus, il est possible d'utiliser les objets détectés comme paramètre de déplacement.

Cas nominal:

Le robot lit correctement l'image, détecte un objet coloré et en détermine les caractéristiques ainsi que la nature (obstacle ou non).

Les résultats sont ensuite utilisés comme contexte pour de futures actions.

Cas dégradé:

1-Image introuvable: L'image d'entrée n'est pas accessible ; le robot renvoie « *Image introuvable* ».

2-Image illisible: Le fichier d'entrée existe mais n'est pas dans un format compatible ; le robot renvoie donc « *Image illisible* ».

3-Objet non détecté : L'image est lisible, mais l'objet recherché n'est pas détecté ; le robot renvoie « *objet recherché non détecté* ».

3. Structure du système

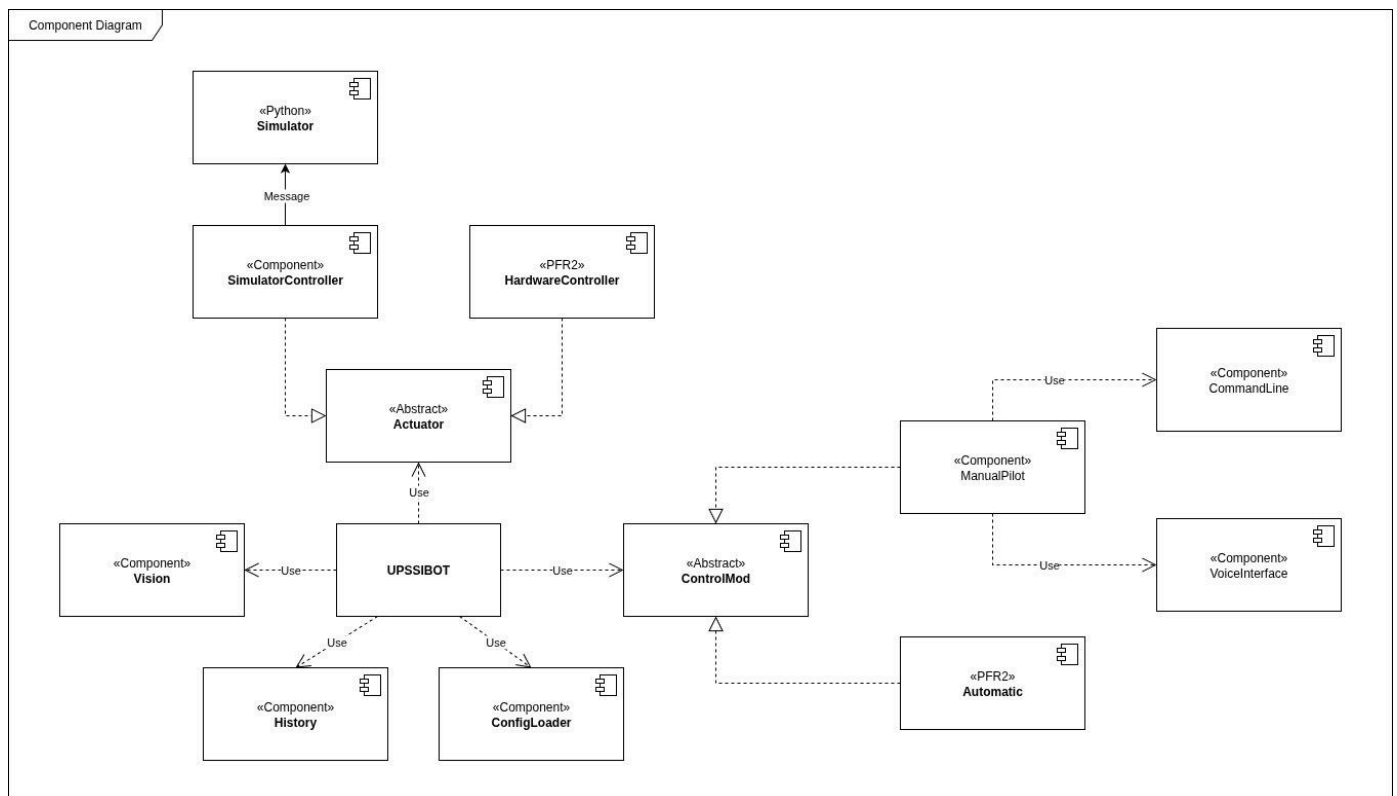


Figure 2: Diagramme de composant de l'Upssibot

La structure globale du système a été modélisée sous la forme d'un diagramme de composants (voir **Figure 2**).

Ce diagramme illustre la manière dont les différents modules logiciels du projet **Upssibot** interagissent entre eux pour remplir les fonctionnalités définies dans le chapitre 2.

Ce tableau présente l'architecture fonctionnelle du robot :

Module	Rôle Principal	Langage	Description
CommandLine	Interaction textuelle	C	Permet la saisie directe de commandes et l'affichage des retours.
VoiceInterface	Interaction vocale	C / Python	Convertit la parole en texte, puis l'interprète en requête.
ManualPilot	Contrôle utilisateur	C	Implémente le contrôle manuel du robot
Simulator	Visualisation graphique	Python (Turtle)	Affiche les déplacements du robot dans un environnement virtuel.
SimulatorController	Communication	C	Envoie les ordres au simulateur

ConfigLoader	Paramétrage	C	Charge les fichiers de configuration (langues, vocabulaires, images, seuil couleurs, etc...).
Vision	Analyse d'image	C	Détecte les objets, formes et couleurs.
History	Traçabilité	C	Enregistre toutes les requêtes, résultats et anomalies.

3.1. Module d'interprétation et d'exécution des commandes

Le module de contrôle manuel **ManualPilot** constitue le lien entre les requêtes de l'utilisateur et les actions physiques du robot. Il s'appuie sur deux interfaces distinctes : une interface en ligne de commande **CommandLine** et une interface vocale **VoiceInterface**, permettant de diversifier les moyens de communication avec le système.

Les commandes issues de ces interfaces sont interprétées, puis transmises au module d'action (**Actuator**), chargé d'exécuter les ordres sous la forme de déplacements ou de mouvements concrets du robot.

3.2. Gestion des actionneurs et environnement de test

Le module **Actuator** définit un comportement abstrait des actionneurs du robot, assurant une compatibilité entre les environnements réels et simulés.

Dans le cadre du projet actuel, le contrôle matériel **HardwareController** est prévu pour une version ultérieure (PFR2), tandis que le **SimulatorController** sera implémenté et a pour objectif de manipuler le module **Simulator** afin de reproduire les comportements du robot dans un environnement Python de test.

3.3. Gestion du traitement d'image

Le module **Vision** est dédié à l'analyse des images fournies. Il permet d'identifier les objets présents dans l'environnement, de reconnaître leurs caractéristiques (forme, couleur, taille) et de déterminer leur rôle potentiel dans le déplacement du robot (obstacle ou non).

Ces informations peuvent être utilisées par les autres modules pour adapter le comportement du robot selon les conditions observées.

3.4. Gestion des paramètres et de l'historique

Le module **ConfigLoader** gère le chargement des paramètres nécessaires au fonctionnement du robot : langues, lexiques, chemins de fichiers, algorithmes et réglages de simulation.

Le module **History** centralise quant à lui les informations liées aux exécutions précédentes :

requêtes traitées, résultats obtenus et éventuelles anomalies rencontrées. Le nom du fichier d'historique produit sera au format «*history-dd-mm-yy_hh-mm.log*» afin de facilement retrouver le fichier correspondant à une exécution. La date utilisée pour le format du fichier est la date de lancement du programme. Aucune limitation de taille ne sera implémentée pour l'instant.

Cette structure permet d'assurer la traçabilité des actions effectuées et facilite les tests ainsi que la validation du système.

4. Structure détaillée du système

Le présent chapitre décrit en détail les fonctionnalités principales du système **Upssibot**, c'est-à-dire l'ensemble des besoins auxquels le projet doit répondre.

Ces besoins fonctionnels sont issus du **diagramme des exigences** (voir **Figure 3 en annexe**), qui a servi de base à la définition du périmètre fonctionnel et à la structuration du système.

4.1. Mode de contrôle

Lors du PFR1 le robot devra nécessairement être contrôlé par un opérateur, cependant lors du PFR2 le robot va gagner la possibilité d'être autonome. Afin de pouvoir faire évoluer le code, le module de contrôle manuel implémentera une spécification abstraite (**ControlMod**) de telle manière qu'il soit interchangeable avec le module de contrôle automatique.

En pilote manuel il est possible d'interagir avec le robot de manière textuelle ou vocale avec deux interfaces: la Command Line Interface (CLI) et l'interface vocal

L'interface CLI permet à l'utilisateur de saisir des commandes via le terminal qui seront par la suite analysées et traduites en instructions exécutables simples (exemple : avance, tourne à droite, etc.).

L'interface vocale quant à elle, permet à l'utilisateur de s'exprimer oralement. Cette expression sera ensuite transcrite en texte via un script fourni en Python. On revient ensuite au cas précédent pour traduire cette instruction en exécutable en prenant en compte tous les synonymes et formulations possibles

4.2. Traitement des requêtes utilisateur

Le module de traitement des requêtes a pour objectif de convertir une requête utilisateur (texte ou voix) en requête-commande exploitable par le robot.

Cette conversion se base sur un vocabulaire configurable et multilingue, défini dans des fichiers de configuration externes.

a) Langues supportées

Le système doit être capable de fonctionner au moins en français et en anglais. La langue utilisée est spécifiée dans un fichier de configuration et peut être changée dynamiquement sans recompilation ou redémarrage. Une nouvelle langue sera implémentée à l'aide d'une configuration du module vocal et de fichiers de vocabulaire spécifiques au langage.

b) Vocabulaire et synonymes

Chaque commande (ex. "avance", "tourne à gauche", "arrête") peut avoir plusieurs synonymes :

“Avance” → “va tout droit”, “va en face”, “move forward”, “go ahead”

“Tourne à gauche” → “à gauche”, “turn left”, “go left”

“Avance jusqu'à la balle rouge” → “atteint la balle rouge”, “fetch the red ball”

Le système doit être capable de reconnaître ces synonymes et d'associer chaque expression à une commande interne unique. L'ensemble du vocabulaire sera défini dans des fichiers de configuration afin de pouvoir facilement changer le comportement. Chaque commande aura au minimum un synonyme.

c) Types de requêtes prises en charge

Les requêtes se déclinent en trois catégories, selon leurs complexités :

Commandes simples : “avance”, “recule”, “tourne à droite”, “arrête”.

Chaque verbe sera associé à une commande qui sera alors utilisée sans paramètres.

Commandes complexes : impliquant des paramètres ou conditions (“avance de deux mètres”, “va jusqu'à la balle rouge”).

Le sujet ainsi que les adjectifs seront utilisés comme paramètres de commandes et nécessiteront un accès au contexte généré par le module **vision** afin d'être exécuté correctement.

Commandes composées : succession d'ordres (“avance puis tourne à droite et va au cube”).

Chaque verbe représente une commande à exécuter. La phrase sera ainsi décomposée en commandes complexes qui seront simplement exécutées dans l'ordre. Les spécifications temporelles telles que “va à la balle bleu *mais d'abord* tourne à droite” ne seront pas supportées et résultera en l'exécution des commandes dans l'ordre des verbes.

d) Gestion des négations

Le système doit également être capable de reconnaître les formes négatives dans les requêtes utilisateur.

Une négation correspond à une inversion de l'action exprimée par la commande, par exemple:

“N'avance pas” → le robot n'effectue aucun déplacement.

“Ne tourne pas à gauche” → le robot ignore la commande.

“Ne bouge plus” → le robot s’arrête immédiatement.

La reconnaissance de la négation repose sur la détection de mots-clés tels que “ne”, “n”, “pas”, “plus”, “jamais” (et leurs équivalents anglais : not, don’t, never).

e) Validation et retour utilisateur

Avant l’exécution, chaque commande est validée.

Si une commande est invalide ou non comprise, le système renvoie un message explicite à l'utilisateur.

Toutes les requêtes et résultats sont enregistrés dans l'historique et le terminal.

4.3. Traitement des images

Le système doit pouvoir analyser une image fournie pour en extraire les objets présents et leurs caractéristiques.

Ces informations sont utilisées pour répondre à des requêtes de type “trouve la balle rouge” ou “avance jusqu’au cube bleu”.

a) Formats et sources d'image

Les images sont lues sous un format texte (.txt) contenant 3 matrices (une pour le niveau de rouge, une pour le niveau de vert et une pour le niveau de bleu), le fichier contient également une ligne d'entête correspondant à la taille des matrices une matrice de couleurs.

b) Couleurs reconnues

Les couleurs à identifier doivent inclure au minimum :

rouge, vert, bleu, jaune, noir et blanc. Chaque canal sera quantifié selon des seuils puis analysés afin d'identifier la couleur qui correspond le mieux.

Ces seuils de couleur sont configurables dans le fichier de paramètre (ex. ajustement des plages de valeurs RGB).

c) Objets détectables

Le système doit être capable de reconnaître des balles, des cubes, ainsi qu'associer un objet comme un obstacle si ce dernier se trouve dans une zone risquant la collision.

La détection des sphères et des cubes utilisera la géométrie euclidienne pour approximer l'origine des objets puis comparera les pixels de l'objet à ce qui serait attendu d'un objet parfait avec cette origine. Un objet sera validé si une proportion donnée de ses pixels correspond. Les seuils de reconnaissance sont configurables dans le fichier de configuration du module de vision.

4.4. Traitement des déplacements

Le système doit simuler les déplacements du robot à partir des commandes interprétées. Pour ce faire des messages sont envoyés à l'environnement virtuel afin d'être exécutés. Si la simulation ne peut pas être jointe, le robot enverra un signal à l'utilisateur afin qu'il choisisse entre arrêter l'exécution ou bien continuer en écrivant seulement dans l'historique.

a) Déplacements simples

Le robot doit pouvoir exécuter les actions suivantes :

avancer, reculer, tourner à droite, tourner à gauche, faire demi-tour, s'arrêter.

b) Déplacements complexes

Le système doit aussi gérer des trajectoires définies par des ordres composés, comme : "trace un carré", "avance jusqu'à la balle rouge puis tourne à gauche", "passe entre la balle rouge et le cube bleu".

c) Gestion des obstacles et objets

Si le module de vision signale un obstacle, le robot doit stopper son mouvement, enregistrer l'événement dans l'historique, puis envoyer un message du type "déplacement impossible – obstacle détecté".

4.5. Simulation

La simulation doit permettre d'observer les commandes en temps réel, de reproduire les trajectoires précédentes grâce à l'historique et de valider la cohérence entre les ordres et les actions exécutées.

La simulation est exécutée sous Linux à l'aide d'un script Python connecté au cœur logique développé en C.

L'environnement simulé est chargé depuis un fichier de carte (.map) qui contiendra les coordonnées de départ de chaque élément. Le nom du fichier utilisé sera configurable. Les couleurs de dessin ainsi que la taille des traits pourront aussi être changées dans le fichier de configuration.

4.6. Contraintes techniques

Le système doit respecter une exigence qualité :

- être développé principalement en langage C11 standard sous environnement Linux,
- pouvoir exécuter des scripts Python pour la simulation et le traitement vocal,
- être configurable sans recompilation,
- respecter une structure modulaire (chaque fonction isolée et testable),
- produire un code propre, standard et commenté, utilisable pour la documentation.
- le système doit analyser les données et prendre une décision en moins de 1000ms

PROJECT DATA					GANTT SCHEDULE																	
TASK	RESPONSABLE	COLLABORATEURS	START DATE	COMPLETION DATE	semaine 1 22/09	semaine 2 29/09	semaine 6/10	semaine 4 13/10	semaine 5 20/10	semaine 6 27/10	semaine 7 03/11	semaine 8 10/11	semaine 9 17/11	semaine 10 24/11	semaine 11 01/12	semaine 12 08/12	semaine 13 15/12	semaine 14 22/12	semaine 15 29/12	semaine 16 05/01	semaine 17 12/01	semaine 18 19/01
Livrable 1 : Dossier de spécifications																						
Lecture et analyse du cahier des charges	TOUS	-	26/09/25	02/10/25	■	■																
Preparation des question pour la reunion 1	ABDELHAFIDH	TOUS	29/09/25	02/10/25		■																
Première rédaction du dossier de spécifications – introduction	ABDELHAFIDH	VICTOR	03/10/25	15/10/25		■	■	■														
Conception du diagramme des exigences	ARTHUS	TOUS	03/10/25	-		■	■	■														
Création du Diagramme de composants	ARTHUS	-	10/10/25	12/10/25			■															
Création du diagramme UML Use Case	VICTOR	ABDELHAFIDH	11/10/25	12/10/25			■															
Rédaction de l'explication du diagramme de composants	ABDELHAFIDH	VICTOR	12/10/25	-			■	■														
Rédaction de l'explication du Use Case Diagram	VICTOR	-	12/10/25	13/10/25			■															
Conception du Diagramme de dépendances	JOAN	-	15/10/25	-				■														
Création du Diagramme de Gantt	ABDELHAFIDH	-	15/10/25	-				■														
Relecture complète du dossier de spéc + mise en page finale	TOUS	-	-	-					■													
Interface utilisateur (5)																						
Choix du mode	ARTHUS	-	27/10/25	05/11/25						■												
CLI (Command Line Interface)	JOAN	-	27/10/25	10/11/25						■	■											
Menu	JOAN	-								■	■											
Interface Vocale	VICTOR	-	27/10/25	10/11/25						■	■											
Traitement de la voix (10)																						
Transcription de l'audio en texte	VICTOR	-	10/11/25	17/11/25								■	■									
Interpretation du texte en requete	VICTOR	-	17/11/25	15/12/25									■	■	■	■						
Support d'autres langues	VICTOR	-	22/12/25	05/01/25										■	■	■		■	■			
Traitement d'image (15)																						
Determination des couleurs	ARTHUS	-	17/11/25	01/12/25									■	■								
Detection de forme : balles	ARTHUS	JOAN	01/12/25	22/12/25											■	■						
Detection de forme : cube	JOAN	ARTHUS	15/12/25	05/01/25												■	■	■	■			
Detection d'obstacles	ARTHUS	-	22/12/25	05/01/25														■	■			
Config (3)																						
Support de la config système	ARTHUS	-	27/10/25	05/11/25						■												
Banque d'image	JOAN	-	10/11/25	17/11/25								■										
Support de la config d'image	JOAN	-	17/11/25	24/11/25									■									
Dictionnaire de vocabulaire (FR)	JOAN	-	24/11/25	08/12/25										■	■							
Dictionnaire multilingue (FR/EN)	JOAN	-	08/12/25	15/12/25												■						
Historique (1)																						
Fonction d'écriture d'une ligne d'historique	ABDELHAFIDH	-	27/10/25	03/11/25						■												
Simulation (7)																						
Réception des messages de l'UPSSIBOT	ARTHUS	-	03/11/25	17/11/25							■	■										
Déplacement du robot simulé	ABDELHAFIDH	-	03/11/25	17/11/25						■	■											
Lecture de fichier carte	ABDELHAFIDH	-	17/11/25	01/12/25									■	■								
Intégration des obstacles	ABDELHAFIDH	-	01/12/25	15/12/25											■	■						
Rejouer en lisant un fichier d'historique	ABDELHAFIDH	-	15/12/25	29/12/25													■	■				
Validation et Test (5)																						
Création de test du simulateur	JOAN	-	05/01/25	12/01/25																	■	
Création de test de l'historique	JOAN	-	12/01/25	19/01/25																	■	
Création de test de config	VICTOR	-	05/01/25	12/01/25																■		
Création de test de vision	ABDELHAFIDH	-	29/12/25	12/01/25																		
Création de test du module vocal	ARTHUS	-	05/01/25	19/01/25																		
Création de test de l'interface utilisateur	ABDELHAFIDH	-	12/01/25	19/01/25																		

